

UFCD 10790 · Projeto de Programação

Sistema de Gestão de Encomendas de Restauro

Rose Maria Restauros

Yasmin Resende

O Problema

1

Gestão manual

Encomendas anotadas em cadernos, folhas soltas ou de memória.

2

Informação perdida

Prazos esquecidos e dúvidas sobre o estado de cada peça.

3

Sem visão do negócio

Nenhuma noção clara da faturação nem da carga de trabalho.

À medida que o número de clientes cresce, o processo manual torna-se insustentável e propício a erros.

A Solução

Uma aplicação de consola que centraliza, num único sítio, toda a gestão do ateliê:

- Registrar clientes e consultar os serviços oferecidos
- Acompanhar cada encomenda do registo até à entrega
- Controlar prazos e identificar trabalhos em atraso
- Ter uma visão clara da faturação e da actividade

O que a aplicação faz

Clientes

Registar e listar clientes

Serviços

Consultar serviços e preços

Encomendas

Criar e gerir os estados

Filtros e atraso

Filtrar e ver trabalhos em atraso

Relatórios

Faturação e contagem por estado

Dados

Guardados em ficheiro JSON

Como foi construída

Python 3 · sem base de dados · dados guardados em ficheiro **JSON**

1

Classes

Cliente, Serviço e Encomenda (POO)

2

Persistência

Guardar e ler os dados em JSON

3

Funções

Uma função por operação

4

Menu

Liga tudo e recebe as escolhas

```
# =====  
# AS CLASSES  
# =====  
  
class Cliente:  
    def __init__(self, nome, contacto):  
        self.nome = nome  
        self.contacto = contacto  
  
class Servico:  
    def __init__(self, nome, preco_base):  
        self.nome = nome  
        self.preco_base = preco_base  
  
class Encomenda:  
    def __init__(self, descricao_pecas, cliente, servico, prazo, preco, estado="Recebida"):  
        self.descricao_pecas = descricao_pecas # ex: "Cómoda antiga em carvalho"  
        self.cliente = cliente # nome do cliente  
        self.servico = servico # nome do serviço escolhido  
        self.prazo = prazo # data limite, ex: "15-07-2026"  
        self.preco = preco # valor a cobrar  
        self.estado = estado # começa sempre em "Recebida"
```

Requisitos cumpridos

11

Requisitos
Funcionais

7

Requisitos Não
Funcionais

- Robustez: o programa não rebenta com entradas inválidas (try/except)
- Integridade: os dados são gravados após cada alteração
- Usabilidade: menu claro e simples de usar
- Compatibilidade: corre em Windows, macOS e Linux

Todos os requisitos definidos no início foram implementados.

Demonstração ao vivo

GESTÃO DE ENCOMENDAS DE RESTAURO

- 1) Registrar cliente
- 2) Listar clientes
- 3) Consultar serviços
- 4) Criar encomenda
- 5) Atualizar estado
- 6) Listar encomendas
- 7) Filtrar encomendas
- 8) Ver encomendas em atraso
- 9) Relatório de facturação
- 10) Relatório por estado
- 0) Sair

Testes e Resultados

Cada funcionalidade foi testada manualmente, incluindo situações de erro.

- ✓ Registrar cliente
- ✓ Atualizar estado
- ✓ Encomendas em atraso
- ✓ Entrada inválida tratada
- ✓ Criar encomenda
- ✓ Listar e filtrar
- ✓ Relatórios
- ✓ Dados mantidos após reabrir

Resultado: todos os testes concluídos com sucesso.

Desafios

Formato das datas

Comparar datas como texto dava erros. Solução: validar e converter as datas para as comparar corretamente.

Comparação de nomes

Um espaço ou maiúscula a mais 'escondia' um cliente. Solução: limpar e ignorar maiúsculas na comparação.

Conclusão

Uma aplicação funcional e robusta, que cumpre todos os requisitos e responde a uma necessidade real do negócio.

Próximos passos

- Interface gráfica mais amigável
- Base de dados se o volume crescer
- Exportar relatórios (PDF / folha de cálculo)

Obrigada!